

ID1200: Operating Systems

Assignment 1

Fall 2025

Author(s): Andreas S. Pedersen
Vasudev Karunakaran
Padmini Thirumalai Ananthan Pillai
Date: September 22, 2025

Contents

1	Question 1	1
1.1	Task 1	1
1.2	Task 2	2
1.3	Task 3	2
2	Question 2	3
2.1	Task 1	3
2.2	Task 2	3
2.3	Task 3	4
3	Question 3	5
4	Question 4	7
4.1	Task 1	7
4.2	Task 2	7
4.3	Task 3	8
5	Question 5	9
5.1	Task 1	9
5.2	Task 2	9
6	Question 6	11
6.1	Task 1	11
6.2	Task 2	12
7	Question 7	13
8	Question 8	15
8.1	Task 1	15
8.2	Task 2	16

Question 1

Task 1

What are the main OS distributions today? What do you think are the relationship of an OS and kernel? Please name three main tasks that Operating Systems do.

Solution

An Operating System (OS) acts as an intermediary between a user and the computer hardware, with the primary goals of executing user programs efficiently, making the computer system convenient to use, and utilizing hardware efficiently.

Key OS distributions today include:

- **GNU** (GNU's Not Unix!): Established in 1983
- **Mac OS**: Introduced in 1984
- **Microsoft Windows**: Launched in 1985
- **Linux**: Created in 1991. Popular Linux distros include Ubuntu, Fedora and Debian

From Slide 9 (Lecture1), the three main tasks of an OS are:

1. **Execute user programs in isolation**: This ensures that programs run independently without interfering with each other.
2. **Make the computer system convenient to use**: The OS simplifies interactions with the computer hardware for the user.
3. **Efficiently use computer hardware**: The OS manages resources like CPU, memory, disk, and network to ensure optimal performance and fair distribution among conflicting requests.

These are achieved through:

- Process management
- Memory management
- I/O and device management
- File system management
- Security and protection

What do you think is the relationship between an OS and the kernel?

The kernel is the core program of the operating system that is always running on the computer, managing hardware resources like CPU, memory, and I/O and providing essential services like mode switching between user mode (for applications) and kernel mode (for privileged operations). The operating system (OS) is a broader term that encompasses the kernel but also includes all the system programs (e.g., compilers, shells, daemons) and user interfaces (e.g., GUI, command-line) that build a complete, usable computing environment on top of it.

Task 2

Specify the Linux environment you choose to use for this course. Which OS distribution and kernel version do you have?

Solution

Our chosen Linux environment is Ubuntu 22.04 LTS (Jammy Jellyfish) with kernel version 5.15.0-73-generic.

Task 3

Please specify the commands you used in the previous question for checking the information.

Solution

The commands used are shown below.

Show OS distribution
<code>lsb_release -a</code>

Show kernel version
<code>uname -r</code>

Question 2

Analyze this code that calls `fork()` to create a child process. Try to compile and run this program on your Linux system.

Question 2 - Application

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int pid = fork();
6
7      if (pid == 0) {
8          printf("I'm the child %d\n", getpid());
9      }
10     else {
11         printf("My child is called %d\n", pid);
12         sleep(10);
13     }
14
15     printf("PID%d says goodbye\n", getpid());
16     return 0;
17 }
```

Task 1

Please explain what is the purpose of inspecting the returned value from `fork()`.

Solution

The `fork()` system call creates a new process running the same image as the current one. The new process, called the child, is *forked* from the original process, the parent. In the parent process, `fork()` returns the process ID (pid) of the child.

Inspecting the returned value from `fork()` therefore allows us to identify whether we are in the child process or the parent process.

Task 2

Explain what a zombie process and orphan process mean. In the code, do you see any zombie and orphan process? Explain why.

Solution

Often the parent process will want to obtain information about when one of its child processes terminates, e.g., the return value. If a child process were to be entirely removed from memory

upon termination, there would be nothing left for the parent to investigate. Therefore, the kernel puts the child into a special state. A process in this state is known as a *zombie*. A zombie process is essentially a skeleton of what once was the process, containing basic data structures for retaining useful data. A zombie process waits for its parent to inquire about its status using, e.g., `wait()`. Once its status has been inquired, the child is formally terminated and ceases to exist as a zombie [1].

An orphaned process is a child process whose parent has been terminated without performing `wait()`. In this case, the child is adopted by the `init` process [1] [2].

The application for this question creates a child process with `fork()`, but does not perform a `wait()`. The child is nearly immediately killed after creation. The parent waits for 10 seconds before being terminated. Within those 10 seconds, the child is a zombie process, existing as a skeleton in the hopeful expectation that its parent might inquire about it. Upon termination of its parent, the now orphaned zombie process is adopted by the `init` process, which calls `wait()` and terminates the child.

Task 3

Run the code and check the status of the processes using the `ps` command. What statuses do you see for the parent process? What statuses do you see for the child process?

Solution

The output of the application is shown below. Line 4 is printed by the parent after 10 seconds.

```
> ./q2
1 | My child is called 29734
2 | I'm the child 29734
3 | PID29734 says goodbye
4 | PID29733 says goodbye
```

If we run the `ps` command within 10 seconds after executing the application, we see the parent process and the zombie child process. Executing `ps` after 10 seconds shows no output, as the parent process is terminated and the orphaned child has been adopted and killed by the `init` process.

```
> ps a | grep q2
1 | 29733 pts/11 S+ 0:00 ./q2
2 | 29734 pts/11 Z+ 0:00 [q2] <defunct>
3 | 29740 pts/12 S+ 0:00 grep --color=auto q2
```

Question 3

Analyze the following code and consider the case where this program is executed by a process. Describe the expected memory layout and explain the parts that will be stored in the text, data, heap, and stack section, respectively. Your analysis should include all variables.

Question 3 - Application

```
1  #define N 1024
2
3  double sum;
4  double *a;
5
6  double getSum(double *x) {
7      int i;
8      double s = 0;
9
10     for (i = 0; i < N; i++) {
11         s += x[i];
12     }
13     return s;
14 }
15
16 int main() {
17     int i;
18     a = (double*) malloc(N*sizeof(double));
19
20     for (i = 0; i < N; i++) {
21         a[i] = i;
22     }
23
24     sum = getSum(a);
25     printf("sum = %.2f\n", sum);
26     return 0;
27 }
```

Solution

- **Text Section:** Contains the executable machine code for the `main` and `getSum` functions, including the loop logic for array initialization and summation. This section is read-only and loaded from the executable file, starting at a low virtual address (e.g., `0x400000` in ELF binaries).
- **Data Section:** Stores the string literal `"sum = %.2f\n"` used in `printf` as a read-only constant. This section is minimal since no initialized global variables are defined.
- **BSS Section:** Holds uninitialized global variables, zero-initialized by the OS loader. Includes `double sum` (8 bytes, initialized to 0.0) and `double *a` (8 bytes, initialized to `NULL`), which persists throughout the program and is accessible across functions.

- **Heap Section:** Contains the dynamically allocated array of 1024 **double** values (8192 bytes total) allocated by `malloc(N * sizeof(double))` in `main`. The array is initialized with values 0 to 1023 (e.g., `a[0] = 0.0`, ..., `a[1023] = 1023.0`), and the pointer `a` (in BSS) references this block.
- **Stack Section:** Manages automatic variables and function call frames, growing downward. Includes:
 - In `main`: `int i` (4 bytes) for the initialization loop.
 - In `getSum`: `int i` (4 bytes) for the summation loop, `double s` (8 bytes) to accumulate the sum (≈ 523776.0), and the parameter `double *x` (8 bytes) pointing to the heap array.
 - Temporary frames for return addresses and implicit parameters (e.g., frame pointer). The stack unwinds after each function returns.

Question 4

Task 1

How many processes will be created by the following code (including the parent process)?

Question 4 - Application

```
1  #define N 3
2
3  int main() {
4      for (int i = 0; i < N; i++) {
5          fork();
6          fork();
7      }
8      return 0;
9  }
```

Solution

The number of processes, P will be:

$$P = 4^3 = 64$$

Task 2

How will you verify the number of processes created in this program?

Solution

If we rewrite the code to look like the code below, once we run the program, we count the number of printouts of "END" that appear. That number equates to the number of processes created.

Question 4 - Verification of number of processes created

```
1  #define N 3
2
3  int main() {
4      for (int i = 0; i < N; i++) {
5          fork();
6          fork();
7      }
8      printf("END");
9      return 0;
10 }
```

Task 3

How many processes are created as a function of N ?

Solution

We use recurrence to solve this problem. Consider function $f(N)$ denoting the number of processes given the value N . The recurrence formula would be $f(N) = 4f(N - 1)$, as in each iteration of the loop, 3 additional processes are created (4 processes total) for each existing process. Evidently, $f(1) = 4$ [Base Case]. Thus, by solving the recurrence, we get the formula $f(N) = 4^N$.

Question 5

Task 1

Which of the following may be shared between the parent and child process after forking? Explain your answer.

- Program
- Process ID
- File descriptors
- Program counter

Solution

We will explain the answer for each of them one by one.

- **Program:** It is shared. A program is stored in the text segment. When the child process is created, as it is merely a duplicate of the parent process, the process it runs functionally is from the same program. Thus, both processes will share the program and access it from the same memory according to their program counters. The program is shared as both the child and parent processes run the same program at the corresponding address according to its program counter.
- **Process ID:** It is not shared. When the new child process is created via fork, if the fork value does not return 0 (i.e., `fork() ≠ 0`), that specific value represents the PID of the new child process, which differs from the parent process. Each process must be uniquely identified. If the PID were the same, we could not distinguish a child process from a parent process.
- **File descriptors:** It is shared. The child process is copied from the parent process, but the process is still being run from the same file resources.
- **Program counter:** It is not shared. The immediate program counter of the parent process after forking will be copied to the child process, but it is not shared, as both processes can execute their subsequent commands independently of each other. Consider the scenario where a conditional jump only triggers for the child process. In that case, the parent and child processes would have different program counters.

Task 2

Which of the following may be shared among threads in a multi-threaded process? Explain your answer.

- Registers
- Heap
- Global variables
- Stack

Solution

For reference, the figure [fig. 1] displays how resources and memory space are allocated to each thread.

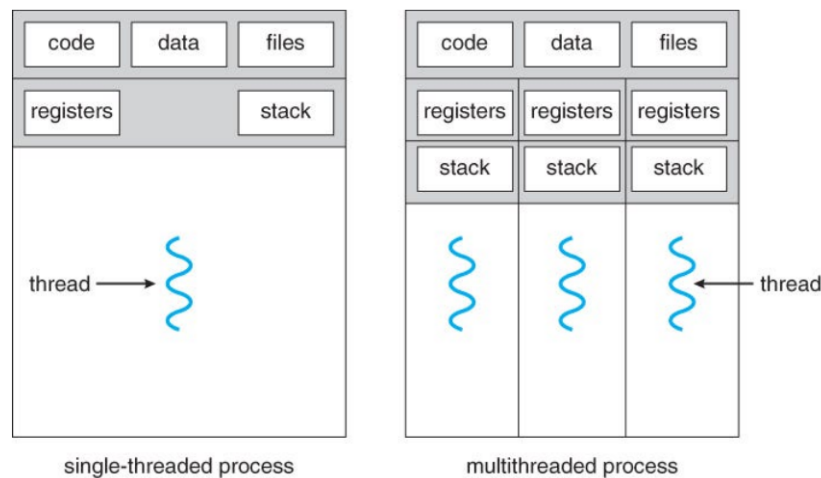


Figure 1: Single-threaded and multithreaded process.

We will explain the answer for each of them one by one.

- **Registers:** They are not shared. Each thread has a corresponding set of registers that get incremented or decremented according to the program code being executed. During context switching, the previous register values are saved into memory. When the previous corresponding process returns to running again, the original register values are reloaded from memory, preventing errors. Imagine a scenario where context switching occurred and process B interrupts process A, then process A returns to running after process B is interrupted. The set of registers for A would be different before and after the switch if the registers were shared. To prevent this, the registers must never be shared.
- **Heap:** It is shared. A heap is stored in memory for dynamic memory allocation, which is accessed by all threads.
- **Global variables:** They are shared. Global variables are stored in the data segment, which is accessed by all threads.
- **Stack:** It is not shared. Each thread has a corresponding stack that changes according to the program code being executed. Imagine a scenario where context switching occurred and process B interrupts process A, then process A returns to running after process B is interrupted. The stack for process A would be different before and after the switch if the stack were shared. To prevent this, the stacks must never be shared.

Question 6

Consider the following application:

```
Question 6 - Application
1  #define N 5
2  int value = 0;
3  void *thread_func(void *param);
4
5  int main(int argc, char *argv[]) {
6      pid_t pid;
7      pthread_t tid;
8      pthread_attr_t attr;
9
10     pid = fork();
11
12     if (pid == 0) {
13         pthread_attr_init(&attr);
14         for (int i = 0; i < N; i++) {
15             pthread_create(&tid, &attr, thread_func, NULL);
16             pthread_join(tid, NULL);
17         }
18         printf("CHILD: value = %d\n", value);
19     }
20     else if (pid > 0) {
21         wait(NULL);
22         printf("PARENT: value = %d\n", value);
23     }
24 }
25
26 void *thread_func(void *param) {
27     value++;
28     pthread_exit(0);
29 }
```

Task 1

What will be the printout of CHILD: value=? Explain your answer.

Solution

The printout will be "CHILD: value=5".

The program uses `fork()` to create a child process. After the fork, both the parent and child processes start with the same initial value of `value`, which is 0. Processes do *not* share global data.

In the child process (where `pid == 0`), the code creates N (5) threads using `pthread_create`. Each thread executes `thread_func`, which increments `value` by 1 and then exits. Since threads within the same process share a single memory space, including global data, the child's global variable, `value`, will be incremented.

The call to `pthread_join` successfully terminates each thread before starting the next. After looping through and creating all $N = 5$ threads, the final `value` of 5 is printed.

Task 2

What will be the printout of `PARENT: value=?` Explain your answer.

Solution

The printout will be "PARENT: value=0".

In the parent process (where `pid > 0`), the `wait(NULL)` call ensures the parent waits for the child process to finish. However, the parent does not create any threads or modify the `value` itself.

Since `value` is a global variable, the parent's copy of `value` is separate from the child's due to process memory isolation after the fork. The child's threads increment its own `value`, but this has no effect on the parent's `value`.

Thus, the parent prints its initial value of 0, which remains unchanged throughout the program's execution in the parent context.

Question 7

Write a C program based on the `fork()` system call. The program will create 2 child processes and perform the following tasks:

- The parent process will create an array of N random numbers whose values are in the range of $[0, 1]$. N is passed by the user via function arguments.
- Child 1 will find the sum of the first half of the array.
- Child 2 will find the sum of the second half of the array.
- The parent will print the sum of the whole array using the results returned by the two child processes.
- Add a timer to time from the process creation till parent printing the final result.

Solution

The proposed solution is found in `applications/src/q7.c`. The parent allocates an array of N elements and fills it with randomly generated values in the range $[0, 1]$. To handle communication between the parent and the children, we use *pipes*, where the children will put the sum of their corresponding half of the array. The application code relating to the first child is shown below:

```

q7.c
1  // child 1, process first half of array
2  pid_t pid_1 = fork();
3  if (pid_1 == 0) {
4      close(pipe_1[0]);
5
6      int sum = 0;
7      for (int j = 0; j < N/2; j++) {
8          sum += arr[j];
9      }
10
11     write(pipe_1[1], &sum, sizeof(sum));
12     close(pipe_1[1]);
13     return 0;
14 }

```

Example output from the application with $N = 1000$ is shown below:

```

./q7.c
1  Sum from Child 1 (first half):    258
2  Sum from Child 2 (second half):  258
3  Parent total sum:                516
4  Total time taken:                773 us

```

To analyze the time taken with as much certainty as possible, a Python script was written that runs the application 1000 times for each value of N and then calculates the average and standard deviation. The application was run on an AMD Ryzen 9950X3D system. The average execution times and standard deviations are summarized in table 1 and fig. 2.

We see that for $N \in \{100, 1000, 10000, 100000\}$, the average execution time is nearly the same. This phenomenon is likely due to the overhead caused by `fork()` (process creation) being the main contributor to execution time. For $N = 1000000$, the execution time is increased ($\sim 786\mu s$ vs. $\sim 264\mu s$), as summing this many numbers requires significantly more computation. The overhead of the arithmetic becomes comparable to the `fork()` overhead.

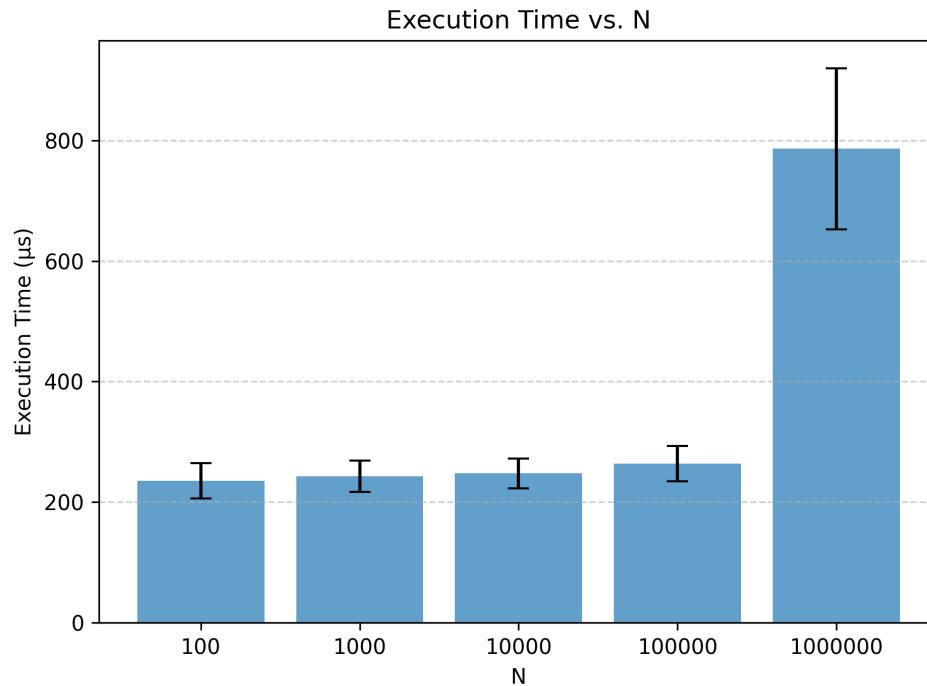


Figure 2: Average times and standard deviations for summing an array of N -size with two subprocesses (children).

N	Average [μs]	Std. Deviation [μs]
100	235.5	29.6
1000	243.0	26.1
10000	247.7	24.7
100000	263.7	29.3
1000000	786.0	133.4

Table 1: Average times and standard deviations for summing an array of N -size with two subprocesses (children).

Question 8

Analyze the following codes for using FIFO for IPC.

Question 8 - Application 1

```
1  int main() {
2      const char *name = "/tmp/myfifo";
3      const char *msg = "hello";
4      int fd;
5
6      mkfifo(name, 0666);
7
8      fd = open(name, O_WRONLY);
9      write(fd, msg, strlen(msg) + 1);
10     close(fd);
11
12     unlink(name);
13
14     return 0;
15 }
```

Question 8 - Application 2

```
1  int main() {
2      const char *name = "/tmp/myfifo";
3      char recv_buf[BUFSIZE];
4      int fd;
5
6      mkfifo(name, 0666);
7
8      sleep(10);
9
10     fd = open(name, O_RDONLY);
11     read(fd, recv_buf, BUFSIZE);
12     close(fd);
13
14     printf("Received %s \n", recv_buf);
15
16     return 0;
17 }
```

Task 1

Identify Producer and Consumer in these two codes.

Solution

We see that Application 1 is the Producer. It opens a FIFO as *write only* (`O_WRONLY`) and writes the string "hello". In contrast, Application 2 is the Consumer. It initializes a receiving buffer and opens a FIFO as *read only* (`O_RDONLY`).

Task 2

Analyse the time dependency among the Producer and Consumer processes. Draw a timeline to illustrate the main function calls including `mkfifo`, `open`, `read/write`, `close`, `unlink`, `print`, in the producer and consumer processes.

Solution

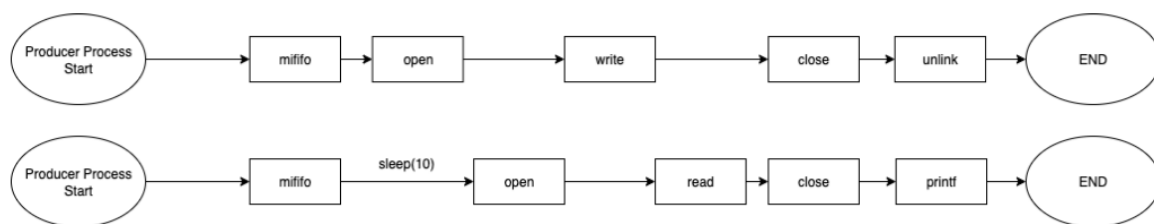


Figure 3: Time dependency among the Producer and Consumer processes.

References

- [1] R. Love, *Linux System Programming: Talking Directly to the Kernel and C Library*. O'Reilly Media, Inc., 2007, ISBN: 0596009585.
- [2] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, 1st. USA: No Starch Press, 2010, ISBN: 1593272200.